



**Universidad
Nacional de
General
Sarmiento**

Informe Mini TP1

Shell y Procesos

López Ciro Martín

Sistemas Operativos y Redes 1

2do cuatrimestre 2025

Profesores: Andres Rojas, Leandro Dikenstein

Informe

En este informe muestro cómo resolví cada uno de los ejercicios, explicando mi razonamiento paso a paso. También comento los problemas que fui encontrando en el camino y las soluciones que apliqué para superarlos. Mi objetivo es reflejar no solo el resultado final, sino también el proceso de aprendizaje que tuve al trabajar en cada punto, además de acompañarlo con imágenes sobre el funcionamiento de los mismos.

Ejercicio 1

Programa: *miniTPEj1.sh*

Planteamiento y resolución

Dividí el problema en dos partes, la primera en resolver fue la sección de la creación de la carpeta en el direccionamiento adecuado y la segunda en la creación de un archivo .txt dentro de la carpeta creada anteriormente y volcar en este el listado pedido.

Al crear la carpeta primeramente lo realice pidiendo que ingrese un nombre como variable el usuario por la terminal, ya que utilicé como guía un ejercicio que estaba de ejemplo en la práctica de Shell. Luego al ver que funcionaba la creación lo cambie para que el usuario ingrese por parámetro, así como indica la consigna.

Para resolver la otra parte no se me complicó tampoco, el comando cat, ls -la ya los había estudiado así que no tuve que investigar cómo realizar esta parte. La parte del mensaje final con lectura de la terminal y cómo redireccionar la salida para volcar el resultado en un archivo también ya la había estudiado, así que no se me presentaron dificultades. Para poder detectar que se ingresó por parámetros un valor lo busqué en internet y me basé en un código de Stack Overflow:

<https://stackoverflow.com/questions/6482377/check-existence-of-input-argument-in-a-bash-shell-script>. Básicamente hice una condición con la flag -z que detecta si la longitud es cero, esta condición está dada al primer argumento pasado. Teniendo esto, si se cumple la condición se ejecuta el mensaje y se cierra el programa y si no, continuamos con el código de abajo.

Problemas

Problemas de permisos: Cuando realice la primera prueba del script, surgió el problema de que el archivo no tenía los permisos necesarios para ser ejecutado. Para resolver este problema no tuve ninguna complicación ya que le asigné los permisos necesarios a mi usuario para poder ejecutar el archivo con el comando: "chmod +x [miniTPEj1.sh](#)", con +x le doy permiso de ejecución. Con esta solución ya no tuve problemas en ejecutar el script directamente.

Problemas de sintaxis: Cuando intentaba volcar el listado de los archivos ubicados en la HOME, me olvidé de poner el signo "\$" delante de "HOME", por lo que no se guardaba el listado de archivos ubicados en el directorio.

Ejercicio 2

Programa: *miniTPEj2.c*

Planteamiento y resolución

Para poder resolver este ejercicio lo primero que hice fue realizar el primer punto de la consigna. Al crear una calculadora ya se estaría cumpliendo con el enunciado, ya que se le pediría al usuario que ingrese los números y se realizan operaciones con estos. Al final, agregué la forma de que el programa se cierre cuando el usuario de enter, esto es útil para tener más control del programa y cuando analicen con htop puedo ver el proceso quieto aunque ya solo queda apretar enter para salir.

Con el punto 2 de la consigna (visualizar el cambio de estado con htop) tuve más problemas ya que no lograba ver el cambio de estado, pero pude solucionarlo ralentizando la parte de resolución algorítmica con un for.



Imagen donde se muestra mi programa en estado S.



Imagen donde se muestra mi programa en estado R.

Problemas

Problemas de sintaxis: Declaré la variable multi como mul, desde la terminal se me mostró este error y como solucionarlo.

Problemas de programación: Al principio solo puse un `getchar()`, por lo que no funcionaba la funcionalidad que quería implementar de esperar a que presione enter para cerrar el programa. Lo resolví agregando otro `getchar()` ya que por Organización del Computador 1 me acordé que se tenía que tener en cuenta el enter ingresado anteriormente que queda guardado en el buffer, por lo que había que capturarlo primero y espera a que ingrese un enter nuevo y ahí capturarlo y cerrar el programa.

Problema en el análisis: Me costó encontrar en `htop` información del programa, así que lo que hice fue hacer un `ps` para ver información del proceso y encontrar el PID del proceso. El comando fue: "`ps au | grep miniTP_Ej2`", la `a` es para mostrar los procesos de todos los usuarios y la `u` para mostrar información detallada después un pipe para filtrar el nombre del programa creado. Después me di cuenta que con `F4` en `htop` podía filtrar y es mucho más fácil encontrar así el programa.

Me surgió el problema que aunque hacia el programa como era pedido, no veía el cambio de estado en `htop`, por lo que tuve que cambiar el código para que tarde más en resolver los problemas y así poder visualizar el cambio de estado del programa en `htop`. Tenía la idea de que al resolverlo tan rápido no llegaba a visualizarse el cambio de estado, por eso esta implementación. Al implementar esta teoría puede visualizar el cambio de estado! no se visualiza en todos los casos, pero si ingresas valores como el 43 si se puede ver.

Ejercicio 3

Programa secuencial: *miniTPEj3.c*

Programa concurrente: *miniTPEj3_2.c*

Planteamiento y resolución

Empecé este ejercicio implementando la función `time()` de la librería con el mismo nombre. No me costó la implementación, simplemente a la función `difftime` le pasé los valores de inicio y fin del programa y esta se encargó de todo. Inicio y Fin los declare como `time_t` y les pasé un valor `time(NULL)`. A inicio le pasé el valor `time(NULL)` antes de realizar los cálculos aritméticos y a fin lo mismo, pero después de los cálculos aritméticos. Logré medir el tiempo de ejecución del programa y el resultado fue de 15 segundos.

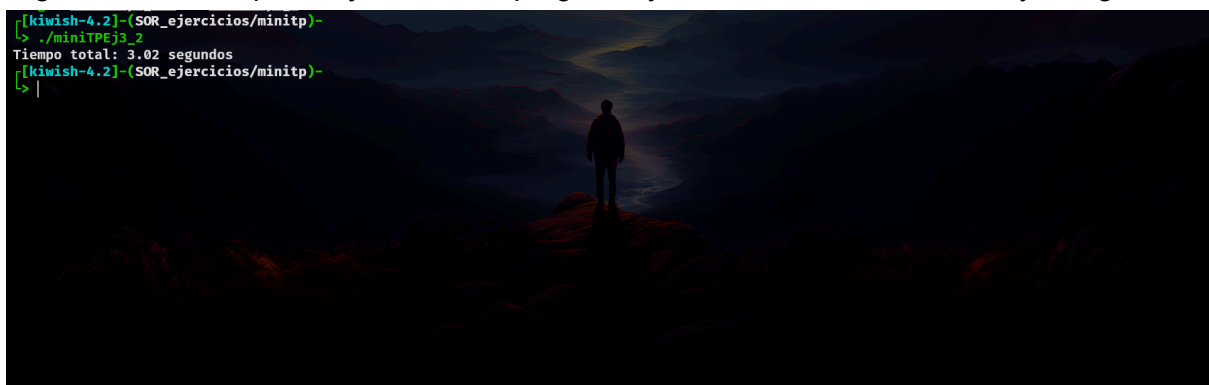
```
[kiwish-4.2]-(SOR_ejercicios/minitp)-
↳ ./minitpej3
El programa tardó 15.00 segundos en ejecutarse.
[kiwish-4.2]-(SOR_ejercicios/minitp)-
↳ |
```



Tiempo de ejecución del programa secuencial.

Luego pasé a la parte de implementar los hilos, para resolverlo utilicé la librería `pthread`. Utilicé `clock_gettime()` para medir el tiempo real de ejecución y `pthread_join()` para esperar a que todos los hilos finalicen para tomar en cuenta el tiempo. También cree hilos con `pthread_create` y utilice `CLOCK_MONOTONIC` como reloj para medir el tiempo ya que es más confiable que usa la hora del sistema que puede cambiar. La implementación de la funcionalidad de calcular tiempo de ejecución es muy parecida a la del programa secuencial, solo está como diferencia el uso de la función `clock_gettime` y que convierto nanosegundos en segundos a la hora de calcular el tiempo total. Para calcular el tiempo total lo que hago es restar los segundos y nanosegundos de inicio y fin para obtener el tiempo, luego convierto los nanosegundos en segundos si corresponde con $/ 1e9$. Logré medir el tiempo de ejecución del programa y el resultado fue de entre 3 y 4 segundos.

```
[kiwish-4.2]-(SOR_ejercicios/minitp)-
↳ ./minitpej3_2
Tiempo total: 3.02 segundos
[kiwish-4.2]-(SOR_ejercicios/minitp)-
↳ |
```



Tiempo de ejecución del programa concurrente.

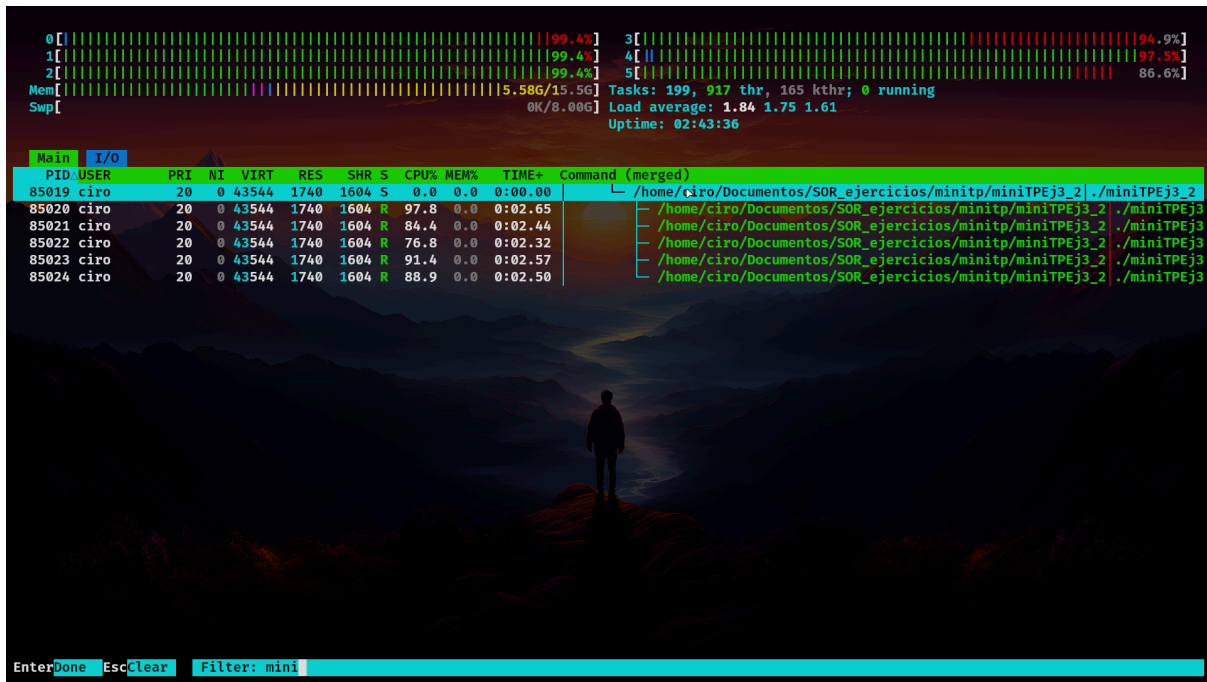


Imagen que muestra mediante htop como el programa se ejecuta en diferentes hilos siendo paralelo.

Problemas

Error en la función de calcular el tiempo de ejecución: Cuando realicé el ejercicio con hilos, noté una mejora significativa en el tiempo de ejecución del programa. Pero por pantalla, se mostraba que los segundos de carga del programa eran disminuidos ínfimamente, dándome como 14.8 segundos siendo que eran mucho menos los que tardaba en la ejecución.

Tras buscar en internet, encontré una consulta en Stack Overflow de alguien que tenía el mismo problema y resulta que era por la mala aplicación de la función clock.

<https://stackoverflow.com/questions/2962785/c-using-clock-to-measure-time-in-multi-threaded-programs>. La función clock suma los tiempos de cada hilo y por eso me marcaba que tardaba casi el mismo tiempo. Al utilizar la función clock_gettime() y el reloj CLOCK_MONOTONIC, obtenemos el tiempo que en verdad tarda el programa.

Uso de hilos: Fue necesario compilar con gcc -pthread para habilitar el soporte de hilos.

¿Por qué es importante la cantidad de núcleos en el procesador?

Cada hilo del programa es asignado a un hilo del kernel. Si el procesador tiene varios núcleos, estos pueden ejecutar varios hilos simultáneamente.

Mi procesador es un i5-9400f con 6 núcleos, por lo que los cinco hilos pueden ejecutarse casi en paralelo, reduciendo drásticamente el tiempo real de ejecución. Si solo hubiera un núcleo, los hilos se intercalarían, sin una mejora real.